Detección de Comunidades en Redes de Colaboración Científica (CDP)

Aroa Gómez Reta
Facultad de Informática
Universidad del País Vasco (UPV/EHU)
agomez382@ikasle.ehu.eus

Udane Saitua Galbarriartu
Facultad de Informática
Universidad del País Vasco (UPV/EHU)
usaitua001@ikasle.ehu.eus

***Resumen*—Este documento presenta la formalización y los primeros experimentos para resolver el Problema de Detección de Comunidades (CDP) en una red de autores del congreso NIPS. Se define el problema bajo el marco PEAS, se detalla la codificación de soluciones y se proponen algoritmos baseline (Búsqueda Aleatoria) para establecer una línea base de rendimiento utilizando la Modularidad como función objetivo. Después, se desarrollan tres algoritmos avanzados para resolver el problema de manera eficiente.**

I. Introducción

El análisis de redes complejas es fundamental para entender la estructura de las colaboraciones científicas. El problema abordado en este trabajo consiste en identificar "comunidades" dentro de un grafo de co-autoría derivado de las publicaciones del congreso NIPS.

Formalmente, dado un grafo $G = (V, E)$, donde V representa a los autores y E a las colaboraciones ponderadas entre ellos, el objetivo es encontrar una partición de V en k grupos tal que la densidad de aristas dentro de los grupos sea significativamente mayor que la densidad de aristas entre grupos. Este es un problema NP-Hard, lo que justifica el uso de métodos heurísticos y metaheurísticos.

II. Definición del Problema

II-A. Formalización del Problema (PEAS)

Para el diseño del agente inteligente que resolverá el problema, utilizamos el esquema PEAS (*Performance, Environment, Actuators, Sensors*):

- **Performance (Medida de Rendimiento):** La calidad de una solución se mide mediante la **Modularidad** (Q). El objetivo es maximizar Q. Un valor alto indica una estructura de comunidad fuerte.
- **Environment (Entorno):** El entorno es estático y totalmente observable. Consta del grafo de colaboración G con $N = 1843$ nodos y sus respectivas aristas ponderadas. El tamaño del espacio de búsqueda viene determinado por el Número de Bell (B_n), que cuenta el número de particiones posibles de un conjunto de n elementos. Se define recursivamente como:

$$B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} B_k \tag{1}$$

  Para nuestro caso de $N = 1843$, el valor de B_{1843} es de una magnitud enorme, lo que hace computacionalmente inviable cualquier método exhaustivo y justifica plenamente el uso de heurísticos de búsqueda estocástica.
- **Actuators (Actuadores):** El agente modifica el estado mediante operadores de vecindad. Las acciones definidas son `generar-vecino-move`, que consiste en cambiar la asignación de comunidad de un autor específico a otra comunidad existente o una nueva; y `generar-vecino-move-majority`, que consiste en cambiar la asignación de comunidad de un autor específico a la comunidad más popular entre los autores de sus nodos vecinos.
- **Sensors (Sensores):** El agente tiene acceso a la matriz de adyacencia del grafo, los grados de los nodos y el valor actual de la función objetivo.

II-B. Codificación de la Solución

El espacio de codificación de las soluciones es N^M, siendo N el número total de autores y M el número de comunidades posibles a asignar. Una solución se representa como un vector S de longitud N:

$$S = [c_0, c_1, \ldots, c_{N-1}] \tag{2}$$

Donde $c_i \in \{0, \ldots, N-1\}$ representa el identificador de la comunidad asignada al nodo i. Cabe destacar que este espacio de codificación presenta redundancia debido a la simetría de las etiquetas (permutar las etiquetas de los grupos no cambia la partición física).

Para definir una solución aleatoria se ha definido la función `generar-particion-random`, que asigna a cada nodo del grafo una comunidad totalmente aleatoria.

II-C. Función Objetivo: Modularidad

La métrica utilizada para evaluar la aptitud (*fitness*) de una solución es la Modularidad de Newman-Girvan, definida matemáticamente como:

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j) \tag{3}$$

Donde:

- A_{ij} es el peso de la arista entre los nodos i y j.
- k_i y k_j son los grados (suma de pesos) de los nodos i y j.
- $m = \frac{1}{2}\sum_{i,j} A_{ij}$ es la suma total de los pesos de las aristas en la red.
- $\delta(c_i, c_j)$ es la delta de Kronecker, que vale 1 si $c_i = c_j$ (están en la misma comunidad) y 0 en caso contrario.

II-D. Definición de Vecindad

Para explorar el espacio de búsqueda, definimos una estructura de vecindad simple. Dada una solución actual S, un vecino S' se genera aplicando un movimiento atómico:

1. Seleccionar un nodo i.
2. Asignar al nodo i una nueva comunidad c_{new} seleccionada aleatoriamente o una nueva vacía (para `generar-vecino-move`) o la comunidad que más famosa sea entre sus vecinos (para `generar-vecino-move`) del conjunto de comunidades activas.

Esto permite al algoritmo transitar entre estados adyacentes en el espacio de búsqueda.

III. Propuesta de implementación

III-A. Algoritmos Baseline

Para establecer una línea base de comparación, se han implementado dos estrategias iniciales:

III-A1. Búsqueda Aleatoria (Random Search): Este algoritmo explora el espacio de búsqueda. Se genera una solución inicial aleatoria y se generan soluciones vecinas aleatorias hasta un número máximo. Se devuelve la partición aleatoria con mejor fitness encontrada. Sirve para verificar que algoritmos más complejos superan al azar.

III-A2. Búsqueda Local (Hill Climbing): Se plantea una estrategia hill climbing básica. Partiendo de una solución inicial, se evalúan vecinos generados por el operador de movimiento. Si un vecino mejora la modularidad actual, se acepta como la nueva solución actual. Este proceso se repite hasta alcanzar un óptimo local.

III-B. Algoritmo 1 (constructivo): greedy randomized

Para la generación de soluciones iniciales de buena calidad, se ha implementado un algoritmo constructivo basado en la heurística de *Propagación de Etiquetas* (Label Propagation). Este enfoque asume que un nodo debe pertenecer a la comunidad con mayor presencia entre sus vecinos directos.

El algoritmo diseñado opera bajo las siguientes premisas:

1. **Inicialización Singleton:** A diferencia de los enfoques que asignan comunidades aleatorias inicialmente, este algoritmo comienza con una partición donde cada nodo $v_i \in V$ pertenece a su propia comunidad única ($C_i = \{v_i\}$). Esto permite que las comunidades crezcan mediante la fusión de nodos, evitando sesgos iniciales artificiales.
2. **Orden de Visita Estocástico:** Para evitar que el orden de los datos en el archivo de entrada condicione la estructura resultante, se genera una permutación aleatoria de los nodos antes de procesarlos.
3. **Regla de Decisión ϵ-Greedy:** Durante la construcción, se recorre cada nodo y se toma una decisión basada en un parámetro de aleatoriedad ϵ (`randomness`):
   - Con probabilidad ϵ (Exploración): Se asigna al nodo una etiqueta de comunidad aleatoria. Esto introduce ruido controlado que ayuda a diversificar las soluciones constructivas y evitar óptimos locales tempranos.
   - Con probabilidad $1 - \epsilon$ (Explotación): Se aplica una regla de *Voto por Mayoría*. El nodo adopta la etiqueta de comunidad que sea más frecuente entre sus vecinos (en caso de empate, se selecciona una de las mayoritarias al azar).

Este método tiene una complejidad computacional baja, cercana a $O(|E|)$, lo que permite generar soluciones base rápidas y razonablemente estructuradas para ser refinadas posteriormente.

III-C. Algoritmo 2 (Búsqueda Local): Simulated Annealing

Para refinar las soluciones y escapar de los óptimos locales, se ha implementado un algoritmo de búsqueda local estocástica basado en la metáfora del Recocido Simulado. Este algoritmo gestiona una única solución actual que evoluciona iterativamente controlada por un parámetro de temperatura T.

III-C1. Operador de Vecindad Híbrido: La definición de `vecino` es crítica en este algoritmo. En lugar de realizar un movimiento totalmente aleatorio, se ha diseñado un operador de mutación híbrido que combina la intensificación (explotación) con la diversificación (exploración).

Dada una partición actual, se selecciona un nodo aleatorio v y se modifica su comunidad siguiendo una política probabilística:

1. **Intensificación por Influencia Local ($80\,\%$ de probabilidad):** El nodo adopta la comunidad de uno de sus vecinos directos en el grafo (elegido al azar). Este movimiento imita la dinámica de propagación de etiquetas, favoreciendo la cohesión local de las comunidades.
2. **Diversificación Estocástica ($20\,\%$ de probabilidad):** El nodo adopta la comunidad de un nodo cualquiera de la red seleccionado aleatoriamente. Este mecanismo introduce ruido, permitiendo saltos a regiones inexploradas del espacio de búsqueda y evitando la convergencia prematura en estructuras locales rígidas.

III-C2. Criterio de Aceptación y Enfriamiento: El algoritmo utiliza el criterio de Metropolis para decidir si acepta una nueva solución vecina S':

$$P(\text{aceptar}) = \begin{cases} 1 & \text{si } \Delta Q > 0 \quad \text{(Mejora)} \\ e^{\Delta Q/T} & \text{si } \Delta Q \leq 0 \quad \text{(Empeora)} \end{cases} \tag{4}$$

Donde ΔQ es la diferencia de modularidad. La temperatura T decrece geométricamente en cada iteración según la fórmula

$T_{k+1} = \alpha \cdot T_k$, con un factor de enfriamiento α (típicamente 0,99 tras la calibración), comenzando desde una $T_{inicial}$ alta que permite una exploración casi aleatoria al inicio del proceso.

III-D. Algoritmo 3 (poblacional): Algoritmo Genético (GA)

Como estrategia poblacional de optimización global, se ha implementado un Algoritmo Genético (GA) generacional con elitismo. Este enfoque mantiene un conjunto de soluciones candidatas que evolucionan simultáneamente mediante operadores inspirados en la selección natural.

III-D1. Representación y Esquema Evolutivo: Cada individuo de la población se codifica como un mapa hash de longitud $N = |V|$, donde el gen i-ésimo representa el identificador de la comunidad asignada al nodo v_i.

El ciclo evolutivo sigue un esquema generacional:

1. **Evaluación:** Se calcula la aptitud (fitness) de cada individuo mediante la función de Modularidad Q.
2. **Elitismo:** Para garantizar la convergencia monótona y no perder las mejores soluciones encontradas, el mejor individuo de la generación t pasa inalterado a la generación $t + 1$.
3. **Reproducción:** El resto de la nueva población se genera mediante selección, cruce y mutación hasta completar el tamaño de la población.

III-D2. Operadores Genéticos: Dadas las características del problema de detección de comunidades, se han seleccionado los siguientes operadores:

- **Selección por Torneo Binario:** Para seleccionar los padres, se escogen dos individuos al azar de la población actual y se selecciona el que posee mayor modularidad. Este método permite ajustar la presión selectiva y es computacionalmente eficiente ($O(1)$).
- **Cruce Uniforme (Uniform Crossover):** A diferencia del cruce de un punto, el cruce uniforme recorre cada gen (nodo) y decide probabilísticamente (con $p = 0{,}5$) si hereda la asignación de comunidad del *Padre A* o del *Padre B*.

$$Hijo[i] = \begin{cases} Padre_A[i] & \text{si } rand(0,1) < 0{,}5 \\ Padre_B[i] & \text{en otro caso} \end{cases} \tag{5}$$

  Esto maximiza la mezcla del material genético, permitiendo que el hijo combine agrupaciones de comunidades exitosas provenientes de ambos progenitores.
- **Mutación Uniforme:** Para mantener la diversidad genética y evitar la convergencia prematura, cada gen tiene una pequeña probabilidad P_m (típicamente $0{,}01 - 0{,}05$) de ser alterado. Si ocurre la mutación, se asigna al nodo una comunidad aleatoria dentro del rango válido $[0, K - 1]$.

IV. Experimentación

IV-A. Algoritmos Baseline

IV-A1. Búsqueda Aleatoria (Random Search): Los resultados obtenidos y el coste temporal para este baseline se pueden observar en las siguientes gráficas:

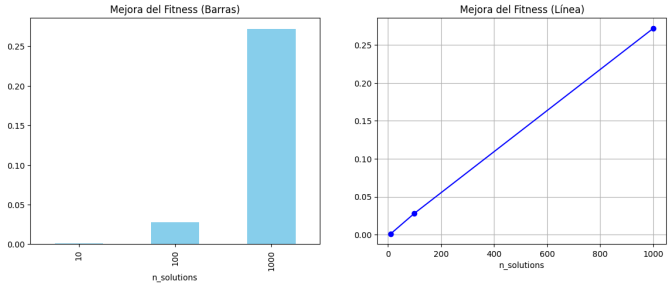

Figura 1. Mejora del fitness Random Search

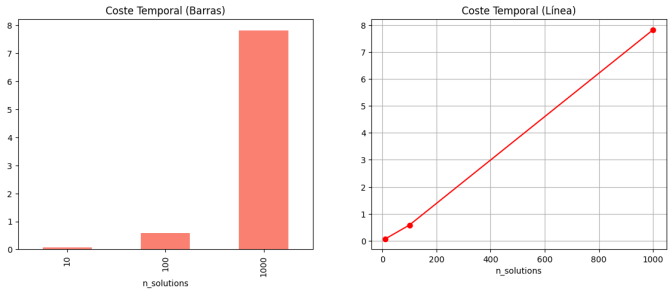

Figura 2. Aumento del coste temporal Random Search

IV-A2. Búsqueda Local (Hill Climbing): Resultados obtenidos:

```
Inicio: 0.11111
Paso 0: Mejora -> 0.50000
Óptimo local alcanzado (ningún nodo quiere moverse).
Size: 6 nodes | Best fitness: 0.5000 | Time: 0.0020s
Inicio: 0.11111
Paso 0: Mejora -> 0.50000
Óptimo local alcanzado (ningún nodo quiere moverse).
Size: 7 nodes | Best fitness: 0.5000 | Time: 0.0005s
Inicio: 0.12245
Paso 0: Mejora -> 0.24490
Óptimo local alcanzado (ningún nodo quiere moverse).
Size: 8 nodes | Best fitness: 0.2449 | Time: 0.0007s
```

Figura 3. Resultados búsqueda local

IV-B. Calibración de hiperparámetros con el framework Optuna

El rendimiento de las metaheurísticas (SA y GA) y del algoritmo constructivo estocástico depende críticamente de la elección adecuada de sus hiperparámetros. Una mala configuración puede llevar a una convergencia prematura o a una exploración ineficiente del espacio de búsqueda.

Para abordar este problema, se ha planteado la calibración como un problema de optimización secundario. En lugar de utilizar una búsqueda en rejilla (*Grid Search*) que resulta costosa computacionalmente, se ha utilizado el framework **Optuna**, el cual implementa algoritmos de optimización bayesiana (concretamente el estimador TPE: *Tree-structured Parzen Estimator*).

El objetivo de la calibración ha sido maximizar la función de fitness (Modularidad Q) sobre el grafo de entrenamiento. Se ha definido un presupuesto computacional limitado para la búsqueda de hiperparámetros antes de lanzar el experimento final.

IV-B1. Algoritmo 1 (constructivo): greedy randomized: Los mejores hiperparámetros encontrados han sido:

- `randomness` = 0,06471094357179763

IV-B2. Algoritmo 2 (Búsqueda Local): Simulated Annealing: Los mejores hiperparámetros encontrados han sido:

- `temp-inicial` = 44,285854246773404
- `alpha` = 0,805154726960718

IV-B3. Algoritmo 3 (poblacional): Algoritmo Genético (GA): Los mejores hiperparámetros encontrados han sido:

- `tam-poblacion` = 48
- `prob-mutacion` = 0,07688379245991422

IV-C. Resultados del diseño experimental

En esta sección se discute el rendimiento comparativo de los cuatro algoritmos implementados. La gráfica muestra la evolución de la calidad de las soluciones (medida mediante la Modularidad Q) en función del número de comunidades objetivo ($n - comunidades$), variando desde 2 hasta 100. Los experimentos se han realizado mediante la ejecución de los diferentes algoritmos con sus hiperparámetros óptimos encontrados por Optuna.

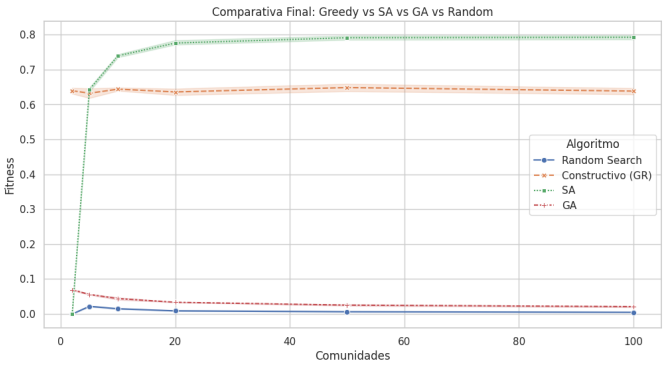

Figura 4. Comparativa de fitness de los diferentes algoritmos en función del número de comunidades

IV-D. Línea Base y Estancamiento (RS y GA)

Tanto la Búsqueda Aleatoria (*Random Search*) como el Algoritmo Genético (GA) muestran un rendimiento prácticamente nulo ($Q \approx 0$) a lo largo de todo el experimento.

- **Random Search:** Este resultado valida la inmensa complejidad del espacio de búsqueda ($N = 1843$ nodos). La probabilidad de encontrar una partición con estructura comunitaria significativa por puro azar es mínima. Actúa correctamente como cota inferior del experimento.
- **Algoritmo Genético:** El estancamiento del GA en valores cercanos a 0 indica una *falta de presión selectiva efectiva*. Dado que la población inicial se genera de forma puramente aleatoria, los individuos carecen de bloques constructivos de calidad. El algoritmo no ha logrado superar la barrera entrópica inicial en las generaciones limitadas del experimento.

IV-D1. Robustez Estructural (Algoritmo Constructivo): El algoritmo Constructivo Greedy (línea superior estable) muestra un buen rendimiento promedio y una notable estabilidad. Su fitness se mantiene consistentemente alto independientemente del número de comunidades $n-comunidades$. Esto se justifica porque el algoritmo explota la **información local del grafo** (estructura de vecindad) de manera voraz. Al guiarse por la mayoría de los vecinos, reconstruye las comunidades naturales del grafo de forma eficaz desde el inicio, sin depender de una búsqueda estocástica a ciegas.

IV-D2. Adaptabilidad Progresiva (Simulated Annealing): Este algoritmo muestra una tendencia ascendente clara: la calidad de la solución mejora a medida que aumenta $n - comunidades$.

- **Para** $n-comunidades$ **bajos:** El algoritmo sufre el efecto de *límite de resolución*, viéndose forzado a fusionar sub-comunidades naturales en pocos grupos grandes, lo que penaliza la modularidad.
- **Para** $n - comunidades$ **altos:** A medida que $n - comunidades$ crece, el SA gana grados de libertad para separar nodos. La pendiente positiva sugiere que la red NIPS posee una estructura de micro-comunidades, y el SA es capaz de encontrarlas progresivamente gracias a su capacidad de escapar de óptimos locales mediante la temperatura.

V. Conclusiones y mejoras

Los resultados evidencian que, para este problema específico, la incorporación de conocimiento del dominio (como hace el algoritmo Constructivo) es superior a la búsqueda genérica (GA/RS) cuando los recursos computacionales son limitados. Sin embargo, el comportamiento ascendente del SA sugiere que las técnicas de búsqueda local tienen potencial si se les permite trabajar con una granularidad de comunidades alta.

Como propuesta de mejora para trabajos futuros, se sugiere implementar una estrategia de **Hibridación (Seeding)** para el Algoritmo Genético. Inyectar soluciones generadas por el algoritmo Constructivo en la población inicial del GA permitiría combinar la capacidad de exploración del evolutivo con la calidad inicial del método voraz, solucionando el problema de estancamiento en 0 observado en este estudio.

Referencias

[1] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," *Physical Review E*, vol. 69, no. 2, p. 026113, 2004.

[2] C. Pizzuti, "GA-Net: A Genetic Algorithm for Community Detection in Social Networks," in *Parallel Problem Solving from Nature – PPSN X*, Springer, 2008, pp. 1081–1090.

[3] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.